

Part 1 Model roles — who does what

● Claude Opus 4.6 — The Architect

System design, complex logic, multi-file reasoning, target formula engine, data architecture, anything requiring deep coherence across the whole codebase. Use when correctness matters more than speed.

Layer 2 logic Data architecture Complex debugging Refactoring

● Claude Sonnet 4.6 — The Builder

Primary coding model. Fast, high quality, handles React components, CSS systems, Three.js scenes, and long files without drifting. Use for 80% of actual code writing.

Component code CSS/Theme files Three.js Screen building Iteration

● Gemini 2.5 Pro High Reasoning — The Reviewer

Best for reviewing what Claude built. Catches logic errors, inconsistencies, performance issues. Also excellent for long-context tasks — feed it your entire codebase for a cross-file audit. Its multimodal capability lets it review screenshots of UI output.

Code review Logic audit UI screenshot review Long context

● Gemini 2.5 Flash — The Grunt

Cheap, fast, disposable tasks. Formatting, renaming, boilerplate generation, converting data formats, writing repetitive code like quote data arrays. Never waste Opus on these.

Data formatting Boilerplate Renaming Quick transforms

● ChatGPT OSS 120B — The Specialist

Use specifically for Three.js and WebGL challenges — its training data has strong coverage of 3D web graphics. Also useful for debugging shader code and particle systems when Claude gets stuck on a 3D-specific issue.

Three.js specifics WebGL shaders 3D math Particle systems

Never use Opus for boilerplate. Never use Flash for architecture. The cost of using the wrong model is wasted context, hallucinated logic, and having to redo the work.

Part 2 MCP skills — honest evaluation

⚠ Reality check: This repo has 1,239 skills. Most are prompt templates, not actual MCP servers. "Skills" here = markdown files that prepend instructions. Useful, but not magic.

● @frontend-design — CRITICAL for your app

Vercel Labs official skill. Enforces design quality, prevents "AI aesthetic" mediocrity. Load this before every UI component you build. It enforces visual hierarchy, spacing, and premium feel constraints.

Use always

● @architecture — CRITICAL for Layer 2

Forces structured thinking before code. Use at the start of each layer to make the model plan before it writes. Prevents the "just start coding" mistake that creates messy, un-refactorable files.

Use at layer start

● @react-patterns — HIGH VALUE

Enforces correct React patterns — no prop drilling, proper hook usage, component isolation. Prevents the most common React mistakes that make code unmaintainable.

Use for Layer 4

- @brainstorming — USEFUL for planning

Forces the model to think before acting. Use when you're about to start a complex new feature and want structured options before committing to one approach.

Use before complex features

- @debugging-strategies — USEFUL when stuck

Systematic debugging framework. When something breaks and you don't know why, this forces methodical isolation instead of random guessing.

Use when stuck

- Most others — LOW VALUE for your use case

Security auditor, DevOps, CI/CD, AWS, pricing-strategy, SEO — all irrelevant for a personal Android app. The 1,239 number is marketing. You need maybe 5-8 skills total.

Ignore

Your actual skill stack: @frontend-design + @architecture + @react-patterns + @brainstorming + @debugging-strategies. That is your complete toolkit.

Part 3 Step-by-step workflow — production quality

- Step 1 — Design token session (Opus)

Before any code: ask Opus to define your complete design system as a tokens.css file. Give it your 5 theme colors, component needs, and spacing requirements. Get it to output the full CSS variable set. This file never changes — everything else inherits from it.

- Step 2 — Logic engine (Opus + @architecture)

Use @architecture skill first to force a plan. Then Opus writes the target formula, streak engine, day boundary logic. Test each function in isolation in the browser console before any UI connects to it. Logic bugs caught here cost nothing. Logic bugs caught in Layer 5 cost hours.

- Step 3 — Component library (Sonnet + @frontend-design + @react-patterns)

Sonnet builds GlassCard, ProgressRing, MalaBead, BottomNav as isolated components with no data. Load @frontend-design before each component prompt. Test each one visually in isolation. A broken component caught alone is a 5-minute fix. A broken component caught inside a screen is a 2-hour hunt.

- Step 4 — 3D scene (ChatGPT 120B + Sonnet)

Use ChatGPT for the Three.js orbit math, particle system setup, and gyroscope input. Then pass the working scene code to Sonnet for React integration and theme connection. Never let one model do both — the 3D math and the React integration have different failure modes.

- Step 5 — Screen assembly (Sonnet)

Sonnet assembles screens by composing the components built in Step 3. It should never be writing new component logic at this stage — only wiring existing pieces together. If it starts inventing new components, stop it and add to the library first.

- Step 6 — Cross-review (Gemini Pro High)

After each layer is complete, feed the entire layer's code to Gemini for review. Also take screenshots of the UI and give them to Gemini for visual quality review. It will catch things Claude missed precisely because it built them.

The workflow is: Plan (Opus) → Build (Sonnet) → Review (Gemini) → Fix (Sonnet) → Next layer. Never skip the review step. It is what separates professional output from amateur output.

Part 4 Advanced prompting frameworks

- For design — the Constraint Framework

Never say "make it look nice." Instead give: (1) what it must NOT look like, (2) a reference aesthetic, (3) specific constraints. Example: "Build a glass card component. Must NOT have any white background, drop shadow, or border-radius below 16px. Reference aesthetic: visionOS spatial UI meets traditional Indian manuscript. Constraints: backdrop-filter blur 20px, border rgba(255,255,255,0.08), inner content must be legible over a dark moving background."

- For motion — the Physics Framework

Describe motion as physics, not as adjectives. Never say "smooth animation." Say: "ease-out cubic-bezier(0.22, 1, 0.36, 1), duration 400ms, starts at scale(0.94) opacity(0), ends at scale(1) opacity(1). No bounce. No spring." Give the model numbers, not feelings.

- For premium UI — the Anti-Template Framework

Before prompting for any UI: explicitly list 5 things you DO NOT want. "Do not use: (1) any shadow with spread > 0, (2) flat colored backgrounds on cards, (3) border-radius below 12px, (4) any animation above 600ms, (5) any color from the standard Tailwind palette." Constraints breed quality. Freedom breeds mediocrity in AI output.

- For 3D — the Scene Description Framework

Describe 3D scenes in cinematography terms, not code terms. "Camera: fixed, FOV 60. Subject: three text instances of रीथ at radii 3, 5, 7 units. Motion: angular velocity 20, -15, 10 degrees/second respectively — they move at different rates and in alternating directions. Emission: custom glow shader, bloom intensity 1.2. Atmosphere: 50 petal particles, slow downward drift 0.02 units/frame."

The single most important prompting principle: be more specific than feels necessary. AI models fill vagueness with defaults. Defaults look cheap. Specificity looks expensive.

Part 5 Why most AI apps look cheap — and how to avoid it

These are the actual failure points — not theoretical. Each one is a specific, avoidable mistake.

- Failure 1 — Default Tailwind colors

The AI defaults to blue-500, gray-200, white. Every AI app has these. Solution: never mention Tailwind in your prompts. Define your own color tokens first and reference only those. Your app uses indigo-900, rose-blush, champak-ivory — not blue-500.

- Failure 2 — Box shadow everywhere

The AI's default "depth" is box-shadow: 0 4px 12px rgba(0,0,0,0.1). This looks like a Bootstrap component from 2018. Solution: forbid box-shadow in your design tokens. Use backdrop-filter blur for depth. Use border with low-opacity white for edge definition. Use glow (box-shadow with color) only sparingly for accent elements.

- Failure 3 — Uniform border-radius

Every element rounded to 8px. It reads as a template. Solution: establish a radius hierarchy — cards 24px, buttons 14px, chips 999px (full pill), input fields 12px. Different elements at different radii create visual rhythm.

- Failure 4 — Animation as decoration

Fade-in on every element, bounce on button click, spin on loading. These signal "AI made this." Solution: motion must communicate state change, not exist for its own sake. Progress ring animates because progress changed. Card enters once on mount, never again. Tap feedback is haptic + scale(0.96), not a color flash.

- Failure 5 — No visual hierarchy

Everything the same size, same weight, same opacity. The eye has nowhere to go. Solution: establish exactly 3 levels of text opacity — primary (100%), secondary (65%), tertiary (38%). Use these consistently. Nothing in between. The eye reads the contrast and knows what matters.

- Failure 6 — Building everything at once

Asking the AI to "build the whole dashboard." It produces a plausible-looking but internally inconsistent mess. Solution: one component at a time, visually verified, before the next. The structural build plan you already have is the solution to this failure.

- Failure 7 — No design system enforcement

The AI invents new values mid-build — a margin of 13px here, a color of #667788 there. After 10 components the design is incoherent. Solution: tokens.css is the single source of truth. Every component references only CSS variables. Never hardcode a color or spacing value in a component file.

The underlying cause of all 7 failures is the same: the AI optimizes for "looks complete" not "looks exceptional." Your job as the director is to raise the bar at every step by being more specific than the model expects.

Part 6 Mapped to your Naam Jap app specifically

- tokens.css → Opus, once, locked forever

All 5 theme palettes as CSS custom properties. Every color, spacing, radius, opacity level defined here. Sonnet never invents values — it only references variables.

- targetEngine.js → Opus + @architecture

The formula is the heart of the app. It gets Opus's full attention. Written once, tested in isolation, never touched again unless the formula changes.

- GlassCard, ProgressRing → Sonnet + @frontend-design

The two most-used components. Built with @frontend-design skill active. Reviewed visually over the actual Sharad Moon background before any screen uses them.

- ThreeScene.jsx → ChatGPT for math, Sonnet for React

The orbit math (spherical coordinates, angular velocity, bloom shader) from ChatGPT. The React component wrapper, theme integration, and gyroscope hook from Sonnet. Two models, two tasks, zero confusion.

- quotes.js → Gemini Flash

All 84 pads from Chaturasi Ji, Sevak Vaani entries, Bhori Sakhi pads structured as a JSON array. Gemini Flash handles this repetitive formatting task — it is exactly what it is built for.

- Visual QA → Gemini Pro High with screenshots

After each screen is built, screenshot it on mobile dimensions and feed to Gemini Pro High. Ask: "Does this look like a premium devotional app or a generic habit tracker? List specific issues." This external review catches what you stop seeing after building.

You already have the right instinct — build structurally, in layers, professionally. This framework just tells you which model to use at each step of that structure.